LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 7: ABSTRACT DAN INTERFACE

Dibuat Oleh :

Nuevalen Refitra Alswando 1030724300008

PROGRAM STUDI INFORMATIKA

FAKULTAS INFORMATIKA

UNIVERSITAS TELKOM SURABAYA

2026

1. TUJUAN PRAKTIKUM

1. Memahami dan mengerti konsep interface dan mampu mengimplementasikannya dalam Java
2. Memahami dan mengerti konsep abstract dan mampu mengimplementasikannya dalam Java
3. Mampu membedakan antara kelas abstract dan interface

2. LANDASAN TEORI

2.1 Kelas Abstract

Kata kunci abstract digunakan untuk method atau class yang belum memiliki implementasi. Abstract method dideklarasikan pada abstract class. Kelas yang dideklarasikan sebagai abstract tidak akan bisa dibentuk Object dalam Java. Kelas abstract tidak dapat digunakan langsung dalam program, kecuali kelas abstract tersebut diturunkan dan turunan dari kelas abstract tersebut yang bisa dijadikan Object.

Karakteristik Abstract Class:

- Tidak dapat diinstansiasi langsung (tidak bisa dibuat objeknya)
- Dapat memiliki method abstract (tanpa body) dan method konkret (dengan body)
- Dapat memiliki atribut dengan berbagai access modifier
- Dapat memiliki constructor
- Subclass wajib mengimplementasikan semua abstract method dari parent class

2.2 Interface

Interface serupa dengan kelas, namun interface mendefinisikan method atau konstanta yang akan diimplementasikan pada Object yang lain. Interface membantu mendefinisikan sifat Object dengan mendeklarasikan seperangkat karakteristik Object tersebut.

Karakteristik Interface (Java 8+):

- Semua atribut adalah public, static, dan final (berperilaku sebagai konstanta)
- Hanya dapat memiliki default method dan static method (selain abstract method)
- Tidak boleh memiliki deklarasi constructor
- Mendukung multiple inheritance (satu class dapat mengimplementasikan banyak interface)
- Method secara default bersifat public dan abstract

2.3 Perbedaan Abstract Class dan Interface

Aspek	Abstract Class	Interface

Method Body	Dapat memiliki method dengan implementasi	Semua method abstract (kecuali default/static di Java 8+)
Atribut	Dapat mendeklarasikan variabel biasa	Hanya konstanta (public static final)
Constructor	Dapat memiliki constructor	Tidak dapat memiliki constructor
Inheritance	Single inheritance	Multiple inheritance
Access Modifier	Semua access modifier diperbolehkan	Hanya public untuk method
Keyword	extends	implements

3. IMPLEMENTASI DAN PEMBAHASAN

3.1 Kelas Abstract - Contoh Employee

Langkah Kerja:

1. Buka NetBeans
2. Buat Project baru: **Modul7_AbstractInterface**
3. Buat package: modul07.abstractclass
4. Buat class abstract **Employee**

File: Employee.java

```java
package modul07.Abstract;

public abstract class Employee {
    private String firstName;
    private String lastName;
    private String socialSecurityNumber;

    // Three-argument constructor
    public Employee(String first, String last, String ssn) {
        firstName = first;
        lastName = last;
        socialSecurityNumber = ssn;
    }

    // Setter dan Getter untuk firstName
    public void setFirstName(String first) { firstName = first; }

    public String getFirstName() {
        return firstName;
    }

    // Setter dan Getter untuk lastName
    public void setLastName(String last) { lastName = last; }

    public String getLastName() { return lastName; }

    // Setter dan Getter untuk socialSecurityNumber
    public void setSocialSecurityNumber(String ssn) { socialSecurityNumber = ssn; // should validate }

    public String getSocialSecurityNumber() { return socialSecurityNumber; }

    // Return String representation of Employee Object
    @Override
    public String toString() {
        return String.format("%s %s\nsocial security number: %s",
                getFirstName(), getLastName(), getSocialSecurityNumber());
    }

    // Abstract method - harus di-override oleh subclass
    public abstract double earnings(); // no implementation here
}
```

Penjelasan:

- Class Employee dideklarasikan sebagai abstract menggunakan keyword abstract
- Memiliki atribut private: firstName, lastName, socialSecurityNumber
- Memiliki constructor untuk inisialisasi atribut
- Memiliki method toString() yang di-override untuk menampilkan informasi employee
- Memiliki **abstract method** earnings() yang tidak memiliki implementasi (body) - method ini WAJIB diimplementasikan oleh subclass

3.2 Subclass dari Abstract Class - SalariedEmployee

File: SalariedEmployee.java

```java
package modul07.Abstract;

public class SalariedEmployee extends Employee {
    private double weeklySalary;

    // Four-argument constructor
    public SalariedEmployee(String first, String last, String ssn, double salary) {
        super(first, last, ssn); // pass to Employee constructor
        setWeeklySalary(salary); // validate and store salary
    }

    // Setter untuk weeklySalary dengan validasi
    public void setWeeklySalary(double salary) { weeklySalary = salary < 0.0 ? 0.0 : salary; }

    // Getter untuk weeklySalary
    public double getWeeklySalary() { return weeklySalary; }

    // Calculate earnings - override abstract method earnings in Employee
    @Override
    public double earnings() { return getWeeklySalary(); }

    // Return String representation of SalariedEmployee Object
    @Override
    public String toString() {
        return String.format("salaried employee: %s\n%s: $%,.2f",
                super.toString(), "weekly salary", getWeeklySalary());
    }
}
```

Penjelasan:

- Class SalariedEmployee mewarisi Employee menggunakan keyword extends
- Memiliki atribut tambahan: weeklySalary
- Constructor memanggil constructor parent menggunakan super()
- Method earnings() **wajib di-override** karena merupakan abstract method di parent class
- Method toString() di-override untuk menampilkan informasi yang lebih spesifik

3.3 Main Class untuk Menguji Abstract Class

File: MainAbstract.java

```java
package modul07.Abstract;

public class MainAbstract {
    public static void main(String[] args) {
        // Employee adalah abstract class, TIDAK BISA dibuat objek langsung
        // Employee emp = new Employee("John", "Doe", "123-45-6789"); // ERROR!

        // Membuat objek dari subclass SalariedEmployee
        SalariedEmployee emp1 = new SalariedEmployee(
                first: "Budi", last: "Santoso", ssn: "111-22-3333", salary: 8000000);

        SalariedEmployee emp2 = new SalariedEmployee(
                first: "Siti", last: "Nurhaliza", ssn: "444-55-6666", salary: 10000000);

        System.out.println("=== DATA KARYAWAN ===\n");

        // Menampilkan informasi employee menggunakan toString()
        System.out.println(emp1.toString());
        System.out.println("Earnings: Rp " + String.format("%,.2f", emp1.earnings()));
        System.out.println();

        System.out.println(emp2.toString());
        System.out.println("Earnings: Rp " + String.format("%,.2f", emp2.earnings()));
        System.out.println();

        // Polymorphism: reference parent, object child
        Employee emp3 = new SalariedEmployee( first: "Ahmad", last: "Rizki", ssn: "777-88-9999", salary: 7500000);
        System.out.println("Polymorphism Example:");
        System.out.println(emp3.toString());
        System.out.println("Earnings: Rp " + String.format("%,.2f", emp3.earnings()));
    }
}
```

```
"C:\Program Files\Java\jdk-25\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3\lib\idea_rt.jar=54442" -Dfile.encoding=UTF-8 -Dsun
=== DATA KARYAWAN ===

salaried employee: Budi Santoso
social security number: 111-22-3333
weekly salary: $8.000.000,00
Earnings: Rp 8.000.000,00

salaried employee: Siti Nurhaliza
social security number: 444-55-6666
weekly salary: $10.000.000,00
Earnings: Rp 10.000.000,00

Polymorphism Example:
salaried employee: Ahmad Rizki
social security number: 777-88-9999
weekly salary: $7.500.000,00
Earnings: Rp 7.500.000,00

Process finished with exit code 0
```

Penjelasan:

- Abstract class Employee **tidak dapat diinstansiasi langsung** (dicomment sebagai contoh error)
- Objek dibuat dari subclass SalariedEmployee yang sudah mengimplementasikan abstract method
- Method earnings() yang di-override dipanggil untuk menghitung penghasilan
- Demonstrasi **polymorphism**: reference bertipe parent (Employee) dapat menunjuk ke objek child (SalariedEmployee)

3.4 Interface - Contoh Payable

Langkah Kerja:

1. Buat package baru: modul07.interfacepkg

2. Buat interface **Payable**

File: Payable.java

```java
package modul07.Interface;

public interface Payable {  2 usages  1 implementation  new *
    // Abstract method - tidak memiliki body
    // Secara default: public abstract double getPaymentAmount();
    double getPaymentAmount(); // calculate payment; no implementation  3 usages  1 implementation  new *
}
```

Penjelasan:

- Interface Payable dideklarasikan menggunakan keyword interface
- Memiliki satu abstract method: getPaymentAmount() yang mengembalikan nilai double
- Method di interface secara default bersifat public dan abstract (tidak perlu ditulis eksplisit)
- Interface tidak memiliki constructor dan tidak dapat diinstansiasi

3.5 Class yang Mengimplementasikan Interface - Invoice

File: Invoice.java

```java
package modul07.Interface;

public class Invoice implements Payable {  5 usages  new *
    private String partNumber;  3 usages
    private String partDescription;  3 usages
    private int quantity;  2 usages
    private double pricePerItem;  2 usages

    // Four-argument constructor
    public Invoice(String part, String description, int count, double price) {  3 usages  new *
        partNumber = part;
        partDescription = description;
        setQuantity(count); // validate and store quantity
        setPricePerItem(price); // validate and store price per item
    }

    // Setter dan Getter untuk partNumber
    public void setPartNumber(String part) { partNumber = part; }

    public String getPartNumber() { return partNumber; }

    // Setter dan Getter untuk partDescription
    public void setPartDescription(String description) { partDescription = description; }

    public String getPartDescription() { return partDescription; }

    // Setter untuk quantity dengan validasi
    public void setQuantity(int count) { quantity = (count < 0) ? 0 : count; // quantity cannot be negative }

    public int getQuantity() {  2 usages  new *
        return quantity;
    }

    // Setter untuk pricePerItem dengan validasi
    public void setPricePerItem(double price) { pricePerItem = (price < 0.0) ? 0.0 : price; // validate price }

    public double getPricePerItem() { return pricePerItem; }

    // Return String representation of Invoice Object
    @Override  new *
    public String toString() {
        return String.format("%s: \n%s: %s (%s) \n%s: %d \n%s: $%.2f"
```

```java
    // Return String representation of Invoice Object
    @Override  new *
    public String toString() {
        return String.format("%s: \n%s: %s (%s) \n%s: %d \n%s: $%,.2f",
                "invoice", "part number", getPartNumber(), getPartDescription(),
                "quantity", getQuantity(), "price per item", getPricePerItem());
    }

    // Method required to carry out contract with interface Payable
    @Override  3 usages  new *
    public double getPaymentAmount() { return getQuantity() * getPricePerItem(); // calculate total cost }
}
```

Penjelasan:

- Class Invoice mengimplementasikan interface Payable menggunakan keyword implements
- **Wajib mengimplementasikan semua abstract method** dari interface (getPaymentAmount())
- Method getPaymentAmount() menghitung total biaya: quantity × pricePerItem
- Memiliki validasi pada setter method untuk memastikan nilai tidak negatif

3.6 Class Tambahan - Product Interface dan Shoe

File: Product.java (Interface)

```java
package modul07.Interface;

public interface Product {  2 usages  1 implementation  new *
    // Konstanta - secara default: public static final
    String MAKER = "MyCorp";  2 usages
    String PHONE = "555-123-4567";  1 usage

    // Abstract method
    int getPrice(int id);  2 usages  1 implementation  new *
}
```

Penjelasan:

- Interface Product memiliki dua konstanta: MAKER dan PHONE
- Memiliki abstract method getPrice(int id) yang harus diimplementasikan oleh class yang mengimplementasikan interface ini

File: Shoe.java (Class yang mengimplementasikan Product)

java

```java
package modul07.Interface;

public class Shoe implements Product {

    @Override
    public int getPrice(int id) {
        if (id == 1)
            return 5;
        else
            return 10;
    }

    public String getMaker() {
        return MAKER; // Mengakses konstanta dari interface
    }
}
```

Penjelasan:

- Class Shoe mengimplementasikan interface Product
- Wajib mengimplementasikan method getPrice(int id) dari interface
- Dapat mengakses konstanta MAKER dari interface secara langsung

3.7 Main Class untuk Menguji Interface

File: MainInterface.java

```java
package modul07.Interface;

public class MainInterface {

    static Shoe hightop;

    public static void init() {
        hightop = new Shoe();
    }

    public static void getInfo(Shoe item) {
        System.out.println("This Product is made by " + item.getMaker());
        System.out.println("It costs $" + item.getPrice(1) + '\n');
    }

    public static void orderInfo(Product item) {
        // Mengakses konstanta dari interface
        System.out.println("To order from " + item.MAKER + " call " + item.PHONE + ".");
        System.out.println("Each item costs $" + item.getPrice(1));
    }
```

```java
    public static void main(String argv[]) {
        init();

        System.out.println("=== INTERFACE EXAMPLE ===\n");

        // Menggunakan Invoice yang mengimplementasikan Payable
        Invoice inv1 = new Invoice( part: "CPU-001", description: "Processor Intel Core i7", count: 2, price: 4500000);
        Invoice inv2 = new Invoice( part: "RAM-002", description: "DDR4 16GB", count: 4, price: 850000);

        System.out.println("Invoice 1:");
        System.out.println(inv1.toString());
        System.out.println("Payment Amount: Rp " + String.format("%,.2f", inv1.getPaymentAmount()));
        System.out.println();

        System.out.println("Invoice 2:");
        System.out.println(inv2.toString());
        System.out.println("Payment Amount: Rp " + String.format("%,.2f", inv2.getPaymentAmount()));
        System.out.println();

        // Menggunakan Shoe yang mengimplementasikan Product
        System.out.println("=== PRODUCT INTERFACE ===");
        getInfo(hightop);
        orderInfo(hightop);

        // Polymorphism dengan interface
        System.out.println("\n=== POLYMORPHISM WITH INTERFACE ===");
        Payable pay1 = new Invoice( part: "MOUSE-003", description: "Wireless Mouse", count: 5, price: 150000);
        System.out.println("Payment via Payable reference: Rp " +
                String.format("%,.2f", pay1.getPaymentAmount()));
    }
}
```

Penjelasan:

- Program menguji implementasi interface Payable melalui class Invoice
- Program menguji implementasi interface Product melalui class Shoe
- Demonstrasi **polymorphism dengan interface**: reference bertipe interface dapat menunjuk ke objek class yang mengimplementasikan interface tersebut
- Konstanta dari interface dapat diakses melalui objek atau nama interface

4. ANALISIS DAN PEMBAHASAN

4.1 Perbandingan Abstract Class vs Interface

Kriteria	Abstract Class	Interface
Instansiasi	Tidak bisa langsung	Tidak bisa langsung
Method Body	Bisa punya method konkret	Hanya abstract (kecuali default/static Java 8+)
Atribut	Variabel biasa dengan semua modifier	Hanya konstanta (public static final)
Constructor	Bisa punya constructor	Tidak bisa punya constructor
Inheritance	Single inheritance (extends)	Multiple inheritance (implements)
Access Modifier	Semua modifier diperbolehkan	Hanya public untuk method
Use Case	"is-a" relationship dengan kode reusable	"can-do" capability / contract

4.2 Kapan Menggunakan Abstract Class?

- Ketika class-class turunan memiliki banyak kesamaan implementasi
- Ketika perlu menyediakan method dengan implementasi default yang bisa di-override
- Ketika perlu mendeklarasikan variabel instance yang bukan konstanta
- Ketika hubungan antar class adalah "is-a" yang kuat

4.3 Kapan Menggunakan Interface?

- Ketika ingin mendefinisikan "capability" atau "contract" yang bisa diimplementasikan oleh class yang tidak berhubungan
- Ketika perlu mendukung multiple inheritance
- Ketika ingin menambahkan method baru tanpa mem-breaking existing code (dengan default method)
- Ketika hubungan antar class adalah "can-do" atau "has-a capability"

4.4 Keuntungan Abstract Class dan Interface

1. **Abstraction**: Menyembunyikan detail implementasi, hanya menampilkan interface
2. **Polymorphism**: Memungkinkan objek diperlakukan sebagai tipe parent/interface
3. **Reusability**: Kode dapat digunakan kembali melalui inheritance/implementasi
4. **Flexibility**: Mudah menambah fungsionalitas baru tanpa mengubah kode existing
5. **Maintainability**: Perubahan pada abstract class/interface otomatis berlaku di subclass/implementasi

5. KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa:

1. **Abstract Class** adalah kelas yang dideklarasikan dengan keyword abstract, tidak dapat diinstansiasi langsung, dan dapat memiliki method abstract yang wajib diimplementasikan oleh subclass.
2. **Interface** adalah kontrak yang mendefinisikan method yang harus diimplementasikan oleh class yang mengimplementasikannya. Interface mendukung multiple inheritance.
3. **Abstract method** adalah method tanpa body yang harus di-override oleh subclass/implementasi class.
4. **Keyword abstract** digunakan untuk class dan method yang belum memiliki implementasi lengkap.
5. **Keyword implements** digunakan untuk mengimplementasikan interface, sedangkan **extends** digunakan untuk mewarisi class (abstract atau konkret).
6. **Java 8+** memperkenalkan default method dan static method pada interface, memungkinkan penambahan method baru tanpa mem-breaking existing code.
7. **Polymorphism** bekerja dengan abstract class dan interface: reference bertipe parent/interface dapat menunjuk ke objek child/implementasi.
8. **Multiple inheritance** hanya dapat dicapai melalui interface, bukan melalui class inheritance.
9. Pemilihan antara abstract class dan interface bergantung pada kebutuhan desain:
    - Gunakan abstract class untuk "is-a" relationship dengan kode reusable
    - Gunakan interface untuk "can-do" capability dan multiple inheritance
10. Pemahaman yang baik tentang abstract class dan interface sangat penting untuk menciptakan desain sistem yang fleksibel, maintainable, dan scalable dalam pemrograman berorientasi objek.

7. DAFTAR PUSTAKA

1. Modul Praktikum Pemrograman Berorientasi Objek Modul 7: Abstract dan Interface
2. Deitel, P.J. & Deitel, H.M. (2017). Java How to Program. Pearson Education.
3. Oracle. (2024). The Java™ Tutorials: Abstract Methods and Classes. https://docs.oracle.com/javase/tutorial/java/IandI/abstract.html
4. Oracle. (2024). Default Methods. https://docs.oracle.com/javase/tutorial/java/IandI/defaultmethods.html